

DIC 2026 Assignment 3

Execution Instructions

Group ID: 68

1 Overview

This document explains how to execute the serverless review processing application for Assignment 3. The application is implemented with MiniStack as an AWS emulator and uses S3, AWS Lambda, DynamoDB, and SSM Parameter Store.

The implemented pipeline is event-driven:

1. A raw review JSON file is uploaded to the raw S3 bucket.
2. The `preprocess` Lambda is triggered by the S3 object creation event.
3. The preprocessed review is written to the preprocessed S3 bucket.
4. The `profanity_check` Lambda is triggered by the new preprocessed object.
5. The profanity result and user ban state are stored/updated using DynamoDB.
6. The profanity-checked review is written to the next S3 bucket.
7. The `sentiment_analysis` Lambda is triggered.
8. The final analyzed review is written to the analyzed S3 bucket and compact result metadata is stored in DynamoDB.

2 Project Structure

The submitted archive contains the following relevant files and directories:

```
src/  
  run.sh  
  requirements.txt  
  process_devset.py  
  lambdas/  
    preprocess/handler.py  
    profanity_check/handler.py  
    sentiment_analysis/handler.py  
  tests/test_integration.py  
  sample_reviews/  
    positive_review.json  
    negative_impolite_review.json
```

3 Prerequisites

The application requires:

- Python 3.11. Python 3.11 is recommended because the Lambda functions are deployed with the `python3.11` runtime.
- MiniStack running locally or on the LBD cluster
- Access to the MiniStack endpoint at `http://localhost:4566`
- Bash shell with standard Unix tools such as `zip`, `find`, `cp`, and `rm`
- Python packages from `src/requirements.txt`

Before running the application, start MiniStack in the used environment. The scripts assume that MiniStack is reachable through:

```
http://localhost:4566
```

4 Setup

Open a terminal in the root directory of the extracted submission archive and enter the source directory:

```
cd src
```

Create and activate a Python virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
```

Install all required dependencies:

```
pip install -r requirements.txt
```

For manual AWS CLI commands, set the following dummy AWS credentials in the current shell:

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1
```

The deployment script `run.sh` also sets these variables internally for the commands it executes.

5 Starting or Restarting MiniStack

Before deploying the application resources, MiniStack must be running. On the LBD cluster, start MiniStack according to the course environment setup. If MiniStack is installed locally, it can typically be started with:

```
ministack start
```

To restart MiniStack locally, stop it and start it again:

```
ministack stop
ministack start
```

After MiniStack has started, verify that the local AWS endpoint is reachable:

```
aws --endpoint-url=http://localhost:4566 s3 ls
```

If the command returns without a connection error, MiniStack is ready. If the AWS CLI reports missing credentials, make sure that the dummy AWS environment variables from the setup section are set in the current shell.

MiniStack is ephemeral. This means that after stopping or restarting MiniStack, all previously created resources are lost. Therefore, the S3 buckets, DynamoDB tables, Lambda functions, SSM parameters, and trigger configurations must be recreated by running the deployment script again:

```
bash run.sh
```

The deployment script and all Python scripts assume the MiniStack endpoint:

```
http://localhost:4566
```

6 Deploying the MiniStack Resources

Run the deployment script:

```
bash run.sh
```

The script creates the following resources:

- S3 buckets: `assignment3-raw-reviews`, `assignment3-preprocessed-reviews`, `assignment3-profanity-checked-reviews`, `assignment3-analyzed-reviews`
- DynamoDB tables: `assignment3-users`, `assignment3-results`
- SSM parameters for all bucket and table names
- Lambda functions: `preprocess`, `profanity_check`, `sentiment_analysis`
- S3 notification triggers connecting the Lambda chain

MiniStack is ephemeral. Therefore, if MiniStack is restarted, the resources must be recreated by running:

```
bash run.sh
```

7 Processing a Single Review

After deployment, a review can be processed by uploading a JSON review file to the raw review bucket.

Example using the provided positive sample review:

```
aws --endpoint-url=http://localhost:4566 s3 cp \  
sample_reviews/positive_review.json \  
s3://assignment3-raw-reviews/positive_review.json
```

Example using the provided negative and impolite sample review:

```
aws --endpoint-url=http://localhost:4566 s3 cp \  
sample_reviews/negative_impolite_review.json \  
s3://assignment3-raw-reviews/negative_impolite_review.json
```

The final analyzed review can be retrieved from the analyzed review bucket:

```
aws --endpoint-url=http://localhost:4566 s3 cp \  
s3://assignment3-analyzed-reviews/positive_review.json -
```

8 Running the Integration Test

The integration test verifies the main pipeline functionality:

- preprocessing
- profanity detection
- sentiment analysis
- counting impolite reviews
- banning a user after more than three impolite reviews

First deploy the resources:

```
bash run.sh
```

Then run the test:

```
pytest tests/test_integration.py
```

9 Processing the Provided Development Set

To process the full `reviews_devset.json` through the deployed MiniStack pipeline, place the file so that it is reachable from the `src` directory and run:

```
python process_devset.py path/to/reviews_devset.json
```

For a faster test run with only a subset of reviews:

```
python process_devset.py path/to/reviews_devset.json --limit 100
```

The script clears the previous local MiniStack state for the application buckets and DynamoDB tables before uploading the development set. It then uploads the reviews to the raw S3 bucket, waits until the Lambda chain has processed them, scans the result tables, and writes the final result summary to:

```
devset_results.json
```

10 Development Set Results

The produced result file contains the following values for the provided development set:

```
{  
  "processed_reviews": 78829,  
  "positive_reviews": 66943,  
  "neutral_reviews": 5332,  
  "negative_reviews": 6554,  
  "failed_profanity_reviews": 3684,  
  "banned_users": [  
    "A320TMDV6KCFU"  
  ]  
}
```